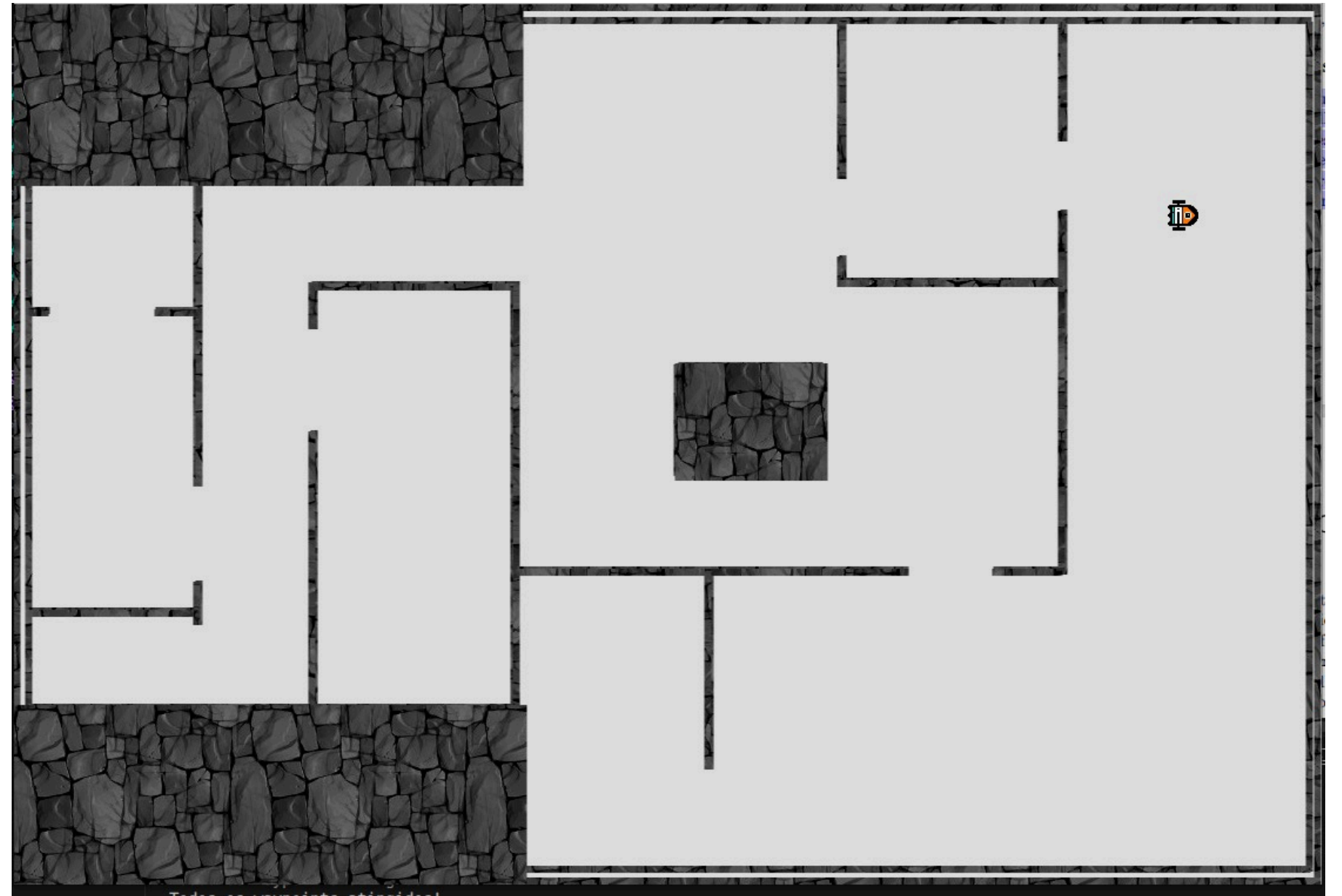
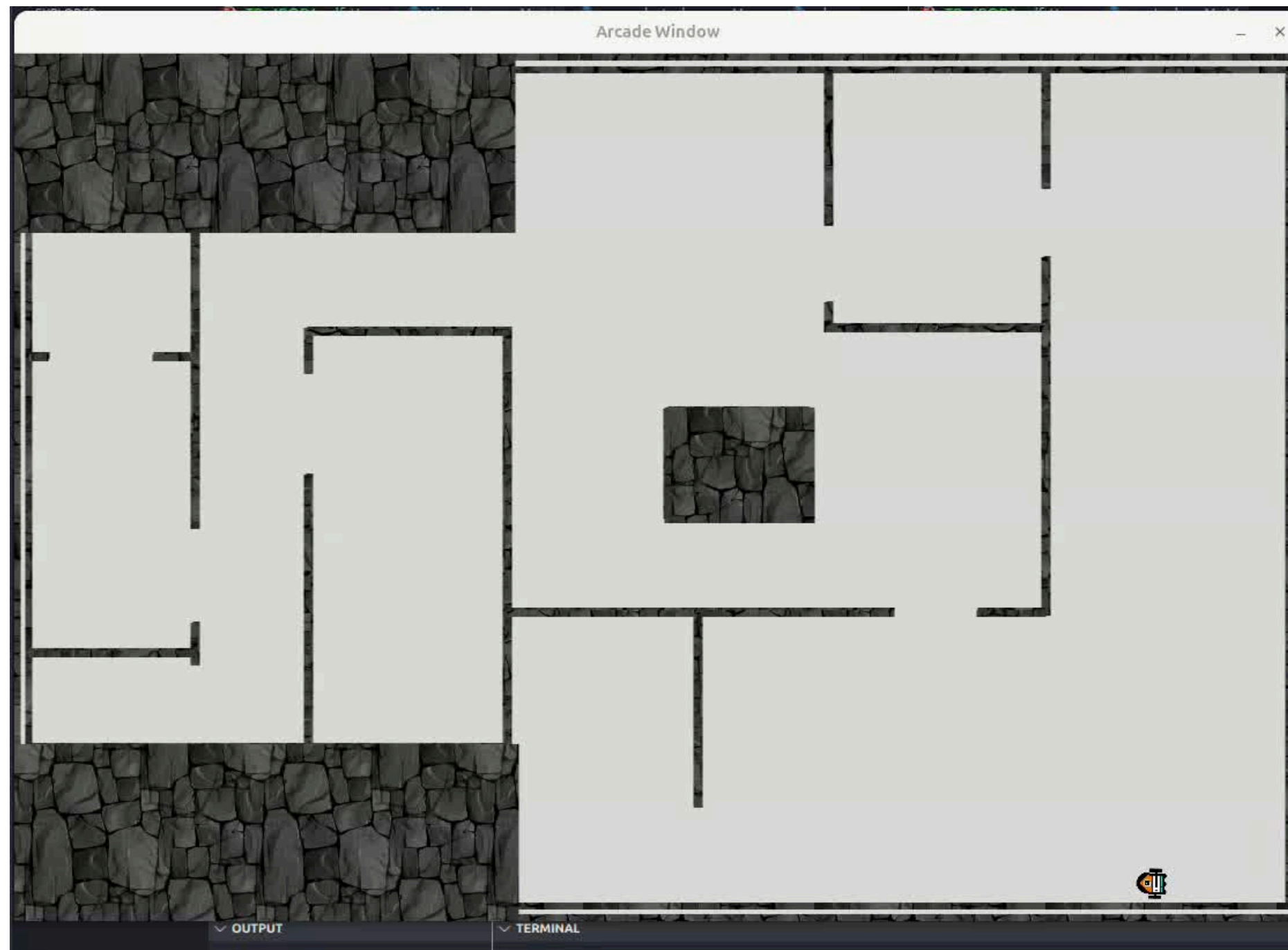


Présentation des travaux réalisés

Naomy Isabela GOMES BIAGGI



Séance 01



1. Définition de la Zone de Détection Frontale

- Concentre sur un "cône" défini par un angle de ± 0.5 rad (environ 30 degrés).
- Une distance minimale de proximité est fixée à 30.

2. Prise de Décision Intelligente (Extension)

- Gauche (angles positifs) et droit (angles négatifs).
- Le robot compare les deux moyennes et tourne délibérément vers le côté qui offre le plus grand espace libre

3. Logique de Navigation à Deux États

- Chemin libre (Comportement par défaut): Si aucun obstacle n'est détecté dans le cône frontal, le robot avance en ligne droite avec une vitesse constante et une rotation nulle.
- Évitement d'obstacle : Si un obstacle est détecté, le robot arrête d'avancer et initie une rotation sur place pour s'échapper.

Séance 02



1. Calcul du champ de gradient

- Fidélité aux formules du TD.
- Dès que le robot est très proche de son objectif, le calcul passe sur un potentiel quadratique.

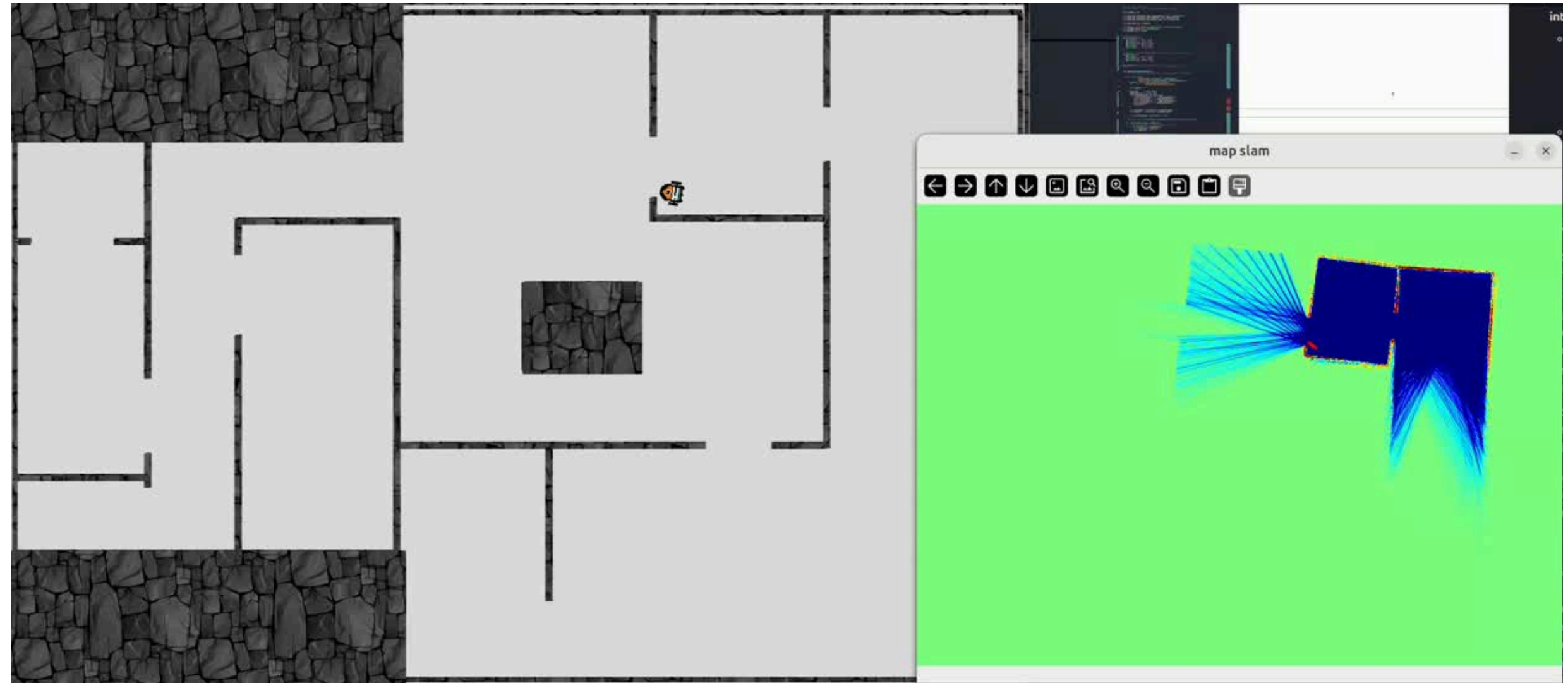
2. Champ Répulsif & Clustering (Extension 1)

- Au lieu de ne prendre que le point le plus proche, l'algorithme regroupe les points LiDAR consécutifs en "clusters" distincts si la distance entre eux est inférieure à un seuil (THRESHOLD = 25).

3. Gestion des Minima Locaux (Extension 2)

- Détection des impasses : L'algorithme vérifie deux conditions critiques : si un obstacle est dangereusement proche, ou si le robot est coincé dans un minimum local (calcul de norme du champ).
- Comportement de récupération (Fallback) : le robot abandonne temporairement le champ de potentiel et fait appel à l'algorithme d'évitement réactif de la Séance 1.

Séance 03



1. Traitement et Conversion des Données LiDAR

- Marge de Sécurité pour garantir la stabilité de la carte et ne pas effacer les obstacles grâce à une erreur.

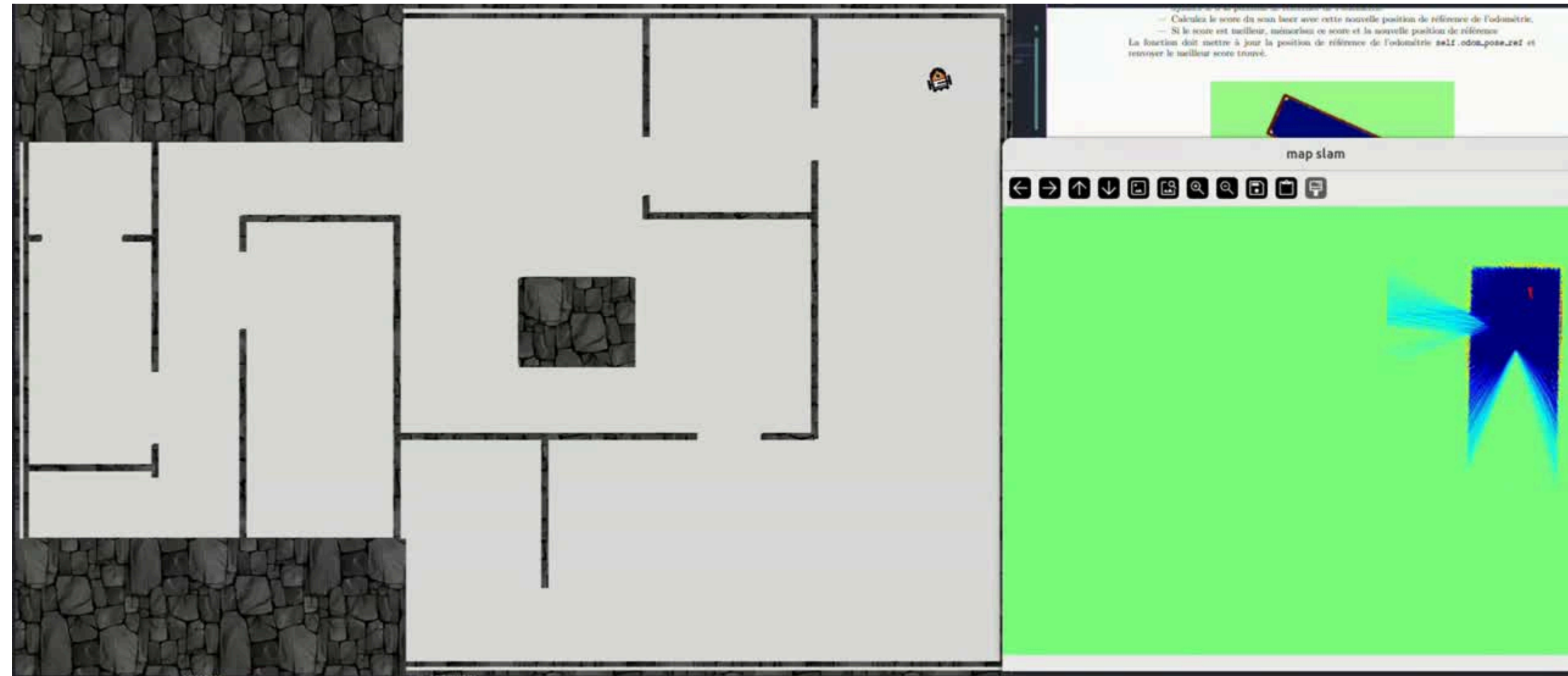
2. Modèle Probabiliste Dynamique (Extension 1)

- Probabilités basées sur la distance : La confiance dans la mesure diminue proportionnellement à la distance .

3. Optimisation des Performances (Extension 2)

- Sous-échantillonnage régulier : l'algorithme ne traite qu'un point LiDAR sur cinq, évitant ainsi de saturer le processeur avec des rayons redondants.
- Filtrage Spatial: Le code utilise **np.unique** sur les coordonnées des cellules de la grille pour filtrer les impacts et ne faire qu'une seule mise à jour par cellule affectée.

Séance 04



1. Fonction de Score

- Sous-échantillonnage : Pour accélérer les calculs sans perdre en précision, seul un point LiDAR sur 5 est conservé.
- Heuristique de Score Améliorée: ajoute une pénalité si les points du laser tombent dans des zones connues pour être vides.

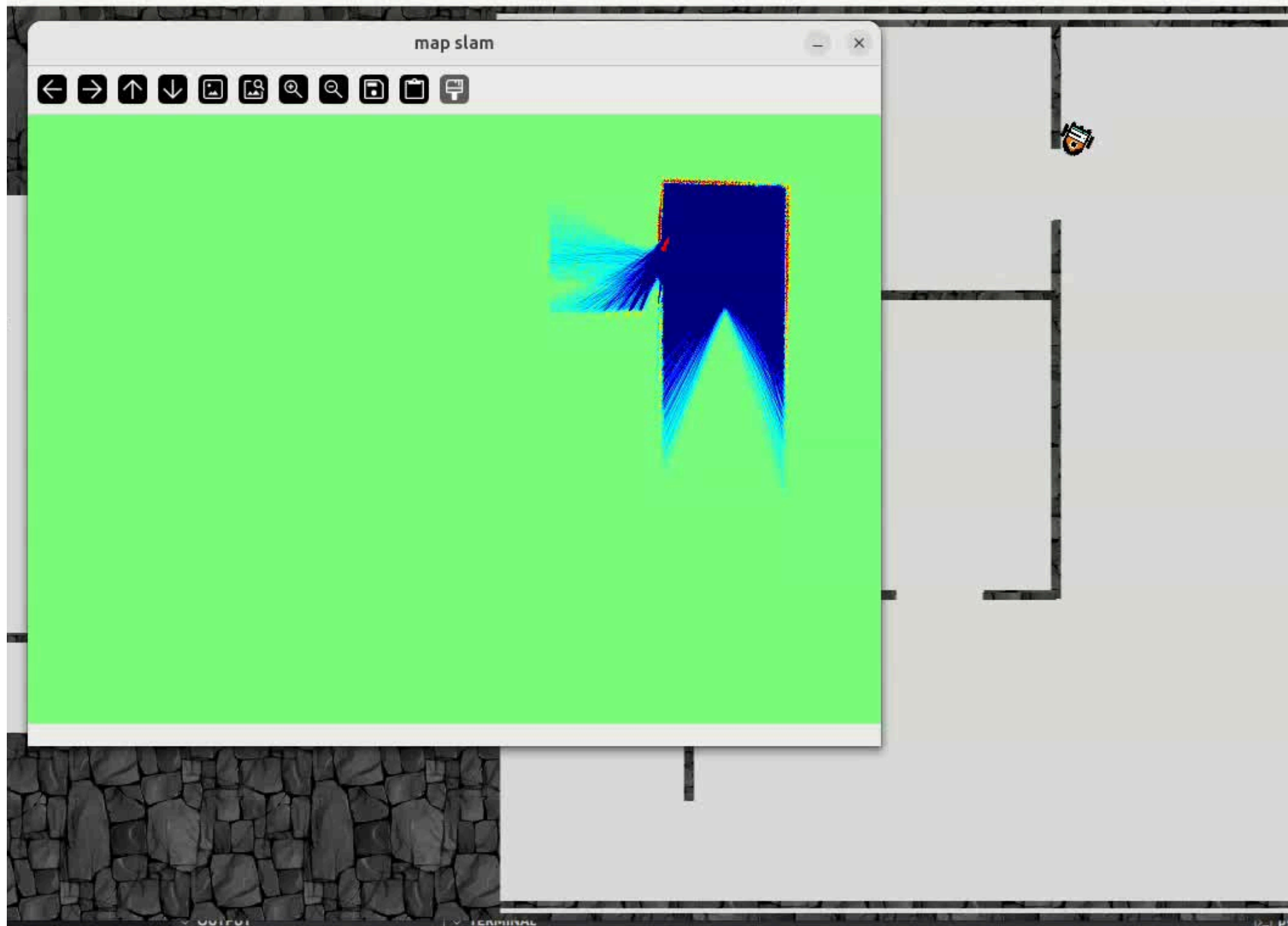
2. Localisation par CEM (Extension)

- Mise en œuvre de la méthode CEM.
- Sélection de l'Élite : À chaque itération, il identifie les 10 meilleures poses.
- La moyenne (mean) du prochain tirage est mise à jour pour correspondre au centre de cette élite

3. Stabilité et Sécurité

- Si la distance entre l'ancienne position et la nouvelle position calculée est trop grand, la mise à jour sera ignorée.
- Seuil de confiance : Si le meilleur score obtenu est jugé catastrophique, l'algorithme conclut qu'il est dans une impasse et préfère conserver sa position précédente plutôt que de compromettre l'intégrité de la carte.

Séance 05



1. Implémentation Fondamentale de A*

- Heuristique Euclidienne : La fonction heuristique calcule la distance en ligne droite entre le nœud courant et la cible.
- L'algorithme utilise une file de priorité (heapq) pour extraire instantanément le nœud le plus prometteur.

2. Sécurité et Robustesse

- Prévention des boucles infinies : Une limite stricte si la cible est physiquement inatteignable.
- L'algorithme cherche la cellule libre la plus proche pour lancer la planification.

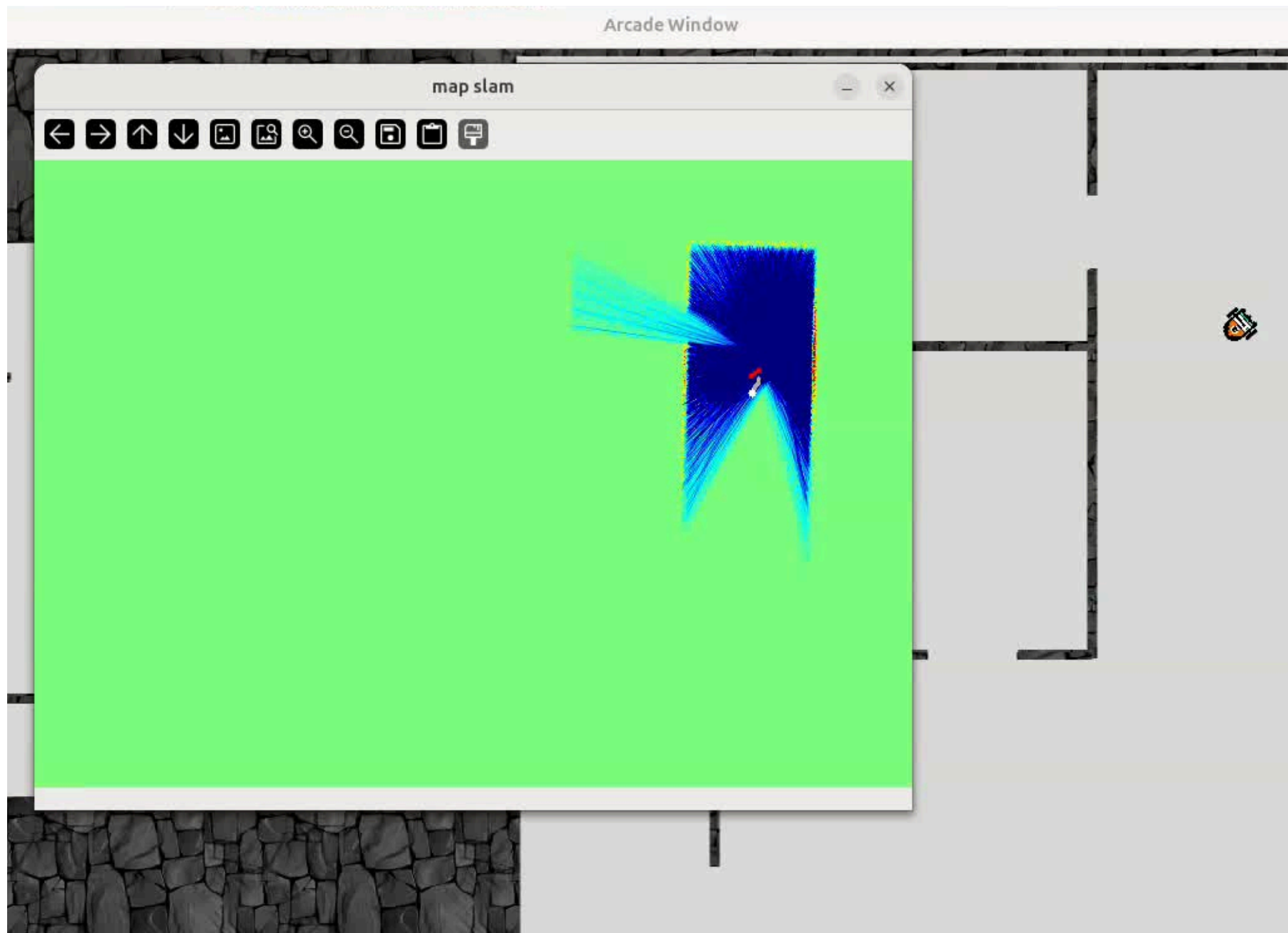
3. Évitement des Obstacles (Extension 1)

- Dilatation Morphologique (Obstacle Inflation) : Pour éviter qu'il ne rase les murs et ne se cogne physiquement.
- L'algorithme A* est donc forcé de générer des trajectoires qui passent au milieu des couloirs.

4. A* Pondéré (Extension 2)

- $U=0$ (Algorithme de Dijkstra) Garantit de trouver le chemin le plus court absolu. Inconvénient : Très lent, car il explore énormément de cellules inutiles.
- $U=1.0$ (A Standard): Avantage : Dirige la recherche vers la cible tout en garantissant le chemin le plus court.
- $U=5.0$ (Glouton) : Avantage : Calcul extrêmement rapide. Inconvénient : Ne garantit plus de trouver le chemin le plus court

Séance 06



1. Détection des Frontières (Frontier-Based Exploration)

- L'algorithme repose sur le principe fondamental de Yamauchi : une "frontière" est la ligne de démarcation entre l'espace connu (libre) et l'espace inconnu (non exploré).
- le code utilise OpenCV (cv2.dilate et cv2.bitwise_and). Il dilate l'espace libre (free_dilated) et cherche l'intersection avec l'espace non exploré (unexplored)

2. Sécurité et Robustesse

- Évitement des murs : Le code identifie les obstacles et les dilate largement.
- Masque des bordures : Ce masque évite que le robot ne sorte des limites physiques du monde.

3. Stratégie de Sélection et Anti-Bouclage

- Heuristique de la distance minimale : Parmi toutes les frontières sûres détectées, l'algorithme parcourt les coordonnées et sélectionne celle qui est euclidiennement la plus proche du robot .
- Mémoire d'exploration : empêche le robot de faire des allers-retours infinis entre deux zones proches mal cartographiées.

4. Machine à États et Retour à la Base

- Le processus est divisé en deux états distincts : EXPLORATION et RETOUR.
- EXPLORATION : Il adopte le comportement réactif de la Séance 1.
- RETOUR : Une fois que la carte est considérée comme complète : après avoir visité 10 frontières ou subi 3 échecs consécutifs de détection), il utilise l'algorithme A* pour planifier et suivre un chemin sécurisé vers son point de départ.